

1. Introduction

This Developer Guide explains how Handy & HandyChat is built, how the code is organized, and how any developer can install, run, maintain, or extend the system.

It is written for:

- CST developers
- Future team members
- Hackathon judges
- Anyone maintaining the system in the future

It includes:

- Setup instructions
- Project structure
- Coding style rules
- How services interact
- How to extend the system
- Testing instructions

2. System Overview

Handy & HandyChat is divided into two components:

Handy (Frontend)

- HTML templates in /templates
- CSS & JS in /static
- Provides PE info, running rules, FAQ, and access to HandyChat

HandyChat (Backend)

- Flask application (app.py)
- Modular routes (routes/)
- Service layer (services/)
- MySQL for PE data
- JSON + PDF for handbook knowledge
- AI engine using GPT-4o-mini

3. Project Structure

HandyChat/

├── app.py

├── config.py

├── extensions.py

├── models.py

├── create_sample_data.py

├── migrate_handbook.py

├── requirements.txt

├── .env

|

├── routes/

| ├── __init__.py

| ├── admin.py

| ├── student.py

| └── unified_chat.py

|

├── services/

| ├── __init__.py

| ├── ai_services.py

| ├── database_services.py

| ├── handbook_pdf_service.py

| ├── handbook_search_service.py

| └── json_handbook_service.py

|

```
|—— static/
|   |—— css/
|   |—— handbook/
|   |—— images/
|   |—— js/
|   └── manifest.json
|
|—— templates/
|   |—— chat.html
|   |—— index.html
|   |—— signup.html
|   └── student.html
|
└── Testing/
    |—— debug_server.py
    |—— reset_database.py
    |—— simple_chat.py
    |—— test_api.py
    |—— test_endpoints.py
    |—— test_flask_connection.py
    |—— test_json_handbook.py
    |—— test_mysql.py
    └── __init__.py
```

4. Setup Instructions

4.1 Prerequisites

- Python 3.8+
- MySQL 5.7+
- pip & virtual environment
- OpenAI API key

4.2 Installation

1) Clone the repo

```
git clone https://github.com/fatima-zouhra/ZJNU_HACKATHON_2025_TEAM10.git  
cd ZJNU_HACKATHON_2025_TEAM10/HandyChat
```

2) Create a virtual environment

```
python -m venv venv
```

3) Activate it

Windows:

```
venv\Scripts\activate
```

Mac/Linux:

```
source venv/bin/activate
```

4) Install dependencies

```
pip install -r requirements.txt
```

4.3 Database Setup

1) Create the database

```
CREATE DATABASE hackathon_team10_db;
```

2) Update .env

```
OPENAI_API_KEY=your_key
```

```
MYSQL_HOST=localhost
```

```
MYSQL_USER=root
```

```
MYSQL_PASSWORD=your_password
```

```
MYSQL_DB=hackathon_team10_db
```

3) Insert sample data

```
python create_sample_data.py
```

4.4 Run the Application

```
python app.py
```

Visit:

```
http://localhost:5000
```

5. Backend Architecture

The backend uses a **3-layer architecture**:

Routes Layer → Services Layer → Data / AI Layer

5.1 Routes Layer

Located in:

/routes

Contains:

- unified_chat.py → main chatbot endpoint
- student.py → student routes
- admin.py → admin or future extension routes

Example (simplified)

```
@unified_chat_bp.route('/ask', methods=['POST'])
```

```
def ask_question():
```

```
...
```

5.2 Services Layer

Located in:

/services

Handles all business logic.

Components:

- **ai_services.py**
 - Intent detection

- Response enhancement
- **database_services.py**
 - Fetch PE class data
 - Join Location + Teacher + Schedule
- **json_handbook_service.py**
 - Fast handbook search from JSON
- **handbook_pdf_service.py**
 - PDF fallback search
- **handbook_search_service.py**
 - Scoring & ranking of handbook results

5.3 Data Layer

MySQL Tables (only PE-related):

- Location
- Teacher
- SportClass
- ClassSchedule

Handbook Source:

- JSON file
- PDF (via pdfplumber)

6. Frontend Architecture

Templates (/templates):

- index.html
- chat.html
- student.html
- signup.html

Static Files (/static):

- CSS
- JS
- Images
- Handbook PDFs

The Chat UI:

- Vanilla JS for sending messages
- Fetch API → /api/chat/ask

7. Environment Variables

From .env:

```
OPENAI_API_KEY=
```

```
MYSQL_USER=
```

```
MYSQL_PASSWORD=
```

```
MYSQL_DB=
```

```
MYSQL_HOST=
```

Handled by:

```
from dotenv import load_dotenv
```

8. Logging

```
logging.basicConfig(level=logging.INFO)
```

```
logger = logging.getLogger(__name__)
```

Used for:

- Debugging
- Error tracking
- Monitoring chatbot flow

9. Testing Guide

API Tests

```
python Testing/test_api.py
```

MySQL Test

```
python Testing/test_mysql.py
```

Flask Connection Test

```
python Testing/test_flask_connection.py
```

Handbook JSON Test

```
python Testing/test_json_handbook.py
```

Endpoint Tests

```
python Testing/test_endpoints.py
```

10. Coding Style Guidelines

✓ Follow these conventions:

- One service per file
- Clear function names (e.g., `get_class_schedule`)
- Use meaningful comments
- Avoid hard-coded credentials
- Use `.env` for secrets
- Keep routes lightweight — logic belongs in services/
- Use consistent error JSON:

```
{ "error": "Invalid input" }
```

11. How to Extend the System

11.1 Add a new sport class

1. Add it to MySQL
2. Update `create_sample_data.py`
3. Add images to `/static/images`

11.2 Add new handbook content

1. Update JSON file

11.3 Add a new route

Create a Python file under:

`/routes`

Register it in `app.py`:

```
app.register_blueprint(...)
```

11.4 Add a new service

Create file in:

`/services`

Import/use it inside `routes/unified_chat.py`.

12. Common Errors & Fixes

! MySQL connection error

- Check .env
- Check MySQL is running
- Confirm DB name exists

! OpenAI API error

- Key missing in .env
- Invalid billing or quota
- Network blocked

! Chatbot gives empty responses

- JSON file path incorrect
- PDF missing in /static/handbook
- AI version not defined

13. Deployment Notes

Local Deployment

Use:

```
python app.py
```

Online Deployment (Future)

Use:

- Railway
- Render
- VPS
- Alibaba Cloud

Make sure to:

- Set environment variables
- Use a production DB
- Move API key to server env
- Enable CORS if needed

14. Conclusion

This Developer Guide provides everything needed for any CST student or future contributor to understand, run, maintain, and extend Handy & HandyChat.

It explains:

- Architecture
- Setup
- Code layers
- Testing
- Extension patterns
- Troubleshooting

This document ensures the project remains maintainable and scalable beyond the hackathon.